

MySQL

COMPLETE REFERENCE GUIDE

Data Types • Constraints • Statements • Clauses • Operators • Aggregate
Functions

Table of Contents

1	Data Types Numeric, String, Date/Time, Spatial & JSON types
2	Constraints Rules enforced on table columns
3	SQL Statements DDL, DML, DCL & TCL command categories
4	Clauses Query-building clauses used with SELECT
5	Operators Arithmetic, comparison, logical & other operators
6	Aggregate Functions Functions that summarize groups of rows
7	Quick Tips & Common Patterns Handy syntax patterns for daily use

SECTION 1

Data Types

1.1 Numeric Data Types

Type	Storage	Range / Notes
TINYINT	1 byte	-128 to 127 (0 to 255 unsigned)
SMALLINT	2 bytes	-32,768 to 32,767
MEDIUMINT	3 bytes	-8,388,608 to 8,388,607
INT / INTEGER	4 bytes	-2,147,483,648 to 2,147,483,647
BIGINT	8 bytes	-9.2 x 10 ¹⁸ to 9.2 x 10 ¹⁸
DECIMAL(M,D) / NUMERIC	Varies	Exact fixed-point; M=total digits, D=decimals. Best for money.
FLOAT(M,D)	4 bytes	Approximate single-precision floating point
DOUBLE(M,D)	8 bytes	Approximate double-precision floating point
BIT(M)	~M/8 bytes	Bit-field value, M from 1 to 64
BOOLEAN / BOOL	1 byte	Alias for TINYINT(1); 0=false, non-zero=true

1.2 String Data Types

Type	Storage	Range / Notes
CHAR(M)	M bytes (padded)	Fixed-length, 0-255 chars; padded with spaces
VARCHAR(M)	Length + 1-2 bytes	Variable-length, up to 65,535 chars
TINYTEXT	Up to 255 bytes	Small text block
TEXT	Up to 65,535 bytes	Standard text block (~64KB)
MEDIUMTEXT	Up to 16,777,215 bytes	Medium text block (~16MB)
LONGTEXT	Up to 4GB	Large text block
BINARY(M)	M bytes	Fixed-length binary string
VARBINARY(M)	Length + 1-2 bytes	Variable-length binary string
BLOB, TINYBLOB, MEDIUMBLOB, LONGBLOB	Varies	Binary large objects (images, files)
ENUM('a','b',...)	1-2 bytes	One value chosen from a predefined list
SET('a','b',...)	1-8 bytes	Zero or more values from a predefined list

1.3 Date & Time Data Types

Type	Format	Range / Notes
DATE	YYYY-MM-DD	1000-01-01 to 9999-12-31
TIME	HHH:MM:SS	-838:59:59 to 838:59:59
DATETIME	YYYY-MM-DD HH:MM:SS	1000-01-01 00:00:00 to 9999-12-31 23:59:59
TIMESTAMP	YYYY-MM-DD HH:MM:SS	1970-01-01 to 2038-01-19; timezone-aware, auto-updates
YEAR	YYYY	1901 to 2155

1.4 Spatial & JSON Data Types

Type	Notes
JSON	Stores JSON documents; validated and stored in optimized binary form
GEOMETRY	Generic spatial value (point, line, polygon, etc.)
POINT	A single (x, y) spatial coordinate
LINESTRING	A curve made of one or more line segments
POLYGON	A spatial polygon shape

SECTION 2

Constraints

Constraints enforce rules on the data allowed in a column, protecting data integrity.

Constraint	Purpose	Example
NOT NULL	Column cannot store a NULL value	name VARCHAR(50) NOT NULL
UNIQUE	All values in the column must be different	email VARCHAR(100) UNIQUE
PRIMARY KEY	Uniquely identifies each row; combines NOT NULL and UNIQUE	id INT NOT NULL PRIMARY KEY
FOREIGN KEY	Links a column to the PRIMARY KEY of another table	FOREIGN KEY (dept_id) REFERENCES dept(id)
CHECK	Ensures values satisfy a specific condition	CHECK (age >= 18)
DEFAULT	Sets a default value when none is provided	status VARCHAR(10) DEFAULT 'active'
AUTO_INCREMENT	Automatically generates a unique number for new rows	AUTO_INCREMENT PRIMARY KEY
INDEX	Speeds up data retrieval (not strictly integrity, but useful)	INDEX(name)

2.1 Referential Actions (used with FOREIGN KEY)

Action	Effect
CASCADE	Automatically updates/deletes matching rows in the child table
SET NULL	Sets the foreign key column to NULL on parent update/delete
RESTRICT	Prevents the delete/update if matching rows exist (default)
NO ACTION	Similar to RESTRICT; checked at statement end
SET DEFAULT	Sets the column to its default value (rarely supported)

SECTION 3

SQL Statements

3.1 DDL - Data Definition Language

Statement	Purpose
CREATE DATABASE / TABLE / INDEX / VIEW	Creates a new database object
ALTER TABLE	Modifies an existing table structure (add/drop/modify columns)
DROP DATABASE / TABLE / INDEX / VIEW	Permanently deletes a database object
TRUNCATE TABLE	Removes all rows from a table quickly, resets AUTO_INCREMENT
RENAME TABLE	Renames one or more tables

3.2 DML - Data Manipulation Language

Statement	Purpose
SELECT	Retrieves data from one or more tables
INSERT INTO	Adds new rows of data into a table
UPDATE	Modifies existing rows in a table
DELETE	Removes existing rows from a table
REPLACE	Inserts a row, replacing it if a duplicate key exists
MERGE (via INSERT...ON DUPLICATE KEY UPDATE)	Inserts or updates in one statement

3.3 DCL - Data Control Language

Statement	Purpose
GRANT	Gives a user specific privileges on database objects
REVOKE	Removes previously granted privileges from a user

3.4 TCL - Transaction Control Language

Statement	Purpose
START TRANSACTION / BEGIN	Marks the starting point of a transaction
COMMIT	Saves all changes made in the current transaction
ROLLBACK	Undoes changes made in the current transaction
SAVEPOINT	Sets a point within a transaction to roll back to
RELEASE SAVEPOINT	Removes a previously defined savepoint
SET AUTOCOMMIT	Enables/disables automatic commit mode

SECTION 4

Clauses

Clauses shape and filter query results, typically used together with SELECT.

Clause	Purpose	Example
WHERE	Filters rows before grouping, based on a condition	WHERE age > 18
GROUP BY	Groups rows sharing a common value for aggregation	GROUP BY department
HAVING	Filters groups after GROUP BY (works with aggregation)	HAVING COUNT(*) > 5
ORDER BY	Sorts the result set (ASC default / DESC)	ORDER BY salary DESC
LIMIT	Restricts the number of rows returned	LIMIT 10 OFFSET 20
JOIN (ON / USING)	Combines rows from two or more tables	JOIN dept ON emp.dept_id = dept.id
UNION / UNION ALL	Combines results of two SELECT queries	SELECT a UNION SELECT b
DISTINCT	Removes duplicate rows from the result	SELECT DISTINCT city
AS	Assigns an alias to a column or table	SELECT name AS emp_name
WITH (CTE)	Defines a named temporary result set	WITH t AS (SELECT ...)
EXISTS	Tests whether a subquery returns any rows	WHERE EXISTS (SELECT 1 ...)

4.1 Types of JOIN

Join Type	Description
INNER JOIN	Returns rows with matching values in both tables
LEFT JOIN (LEFT OUTER JOIN)	All rows from left table + matched rows from right (NULL if none)
RIGHT JOIN (RIGHT OUTER JOIN)	All rows from right table + matched rows from left (NULL if none)
FULL OUTER JOIN	All rows from both tables (emulated in MySQL via UNION of LEFT/RIGHT)
CROSS JOIN	Cartesian product - every row from table A with every row from table B
SELF JOIN	A table joined with itself, using aliases

SECTION 5

Operators

5.1 Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
% or MOD	Modulo (remainder)

5.2 Comparison Operators

Operator	Meaning
=	Equal to
<> or !=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
BETWEEN ... AND ...	Value lies within an inclusive range
IN (...)	Value matches any value in a list
LIKE	Pattern match using % (any chars) and _ (single char)
IS NULL / IS NOT NULL	Tests whether a value is/isn't NULL
<=>	NULL-safe equal to (treats NULL = NULL as true)

5.3 Logical Operators

Operator	Meaning
AND (&&)	True if both conditions are true
OR ()	True if either condition is true
NOT (!)	Reverses a boolean condition
XOR	True if exactly one condition is true

5.4 Other Operators

Operator	Meaning
ALL	Compares a value to all values in a subquery
ANY / SOME	Compares a value to any value in a subquery
:=	Assignment operator (used in variables/UPDATE)
-> and ->>	Extract JSON value (->> unquotes the result)

SECTION 6

Aggregate Functions

Aggregate functions operate on a set of rows and return a single summary value. Typically used with GROUP BY.

Function	Purpose	Example
COUNT()	Counts the number of rows / non-NULL values	COUNT(*), COUNT(DISTINCT city)
SUM()	Adds up numeric values	SUM(salary)
AVG()	Calculates the average of numeric values	AVG(marks)
MIN()	Returns the smallest value	MIN(price)
MAX()	Returns the largest value	MAX(price)
GROUP_CONCAT()	Concatenates values from a group into one string	GROUP_CONCAT(name SEPARATOR ', ')
STDDEV() / STD()	Returns the population standard deviation	STDDEV(score)
VARIANCE()	Returns the population variance	VARIANCE(score)
BIT_AND() / BIT_OR() / BIT_XOR()	Bitwise aggregate operations across a group	BIT_AND(flags)
JSON_ARRAYAGG()	Aggregates values into a JSON array	JSON_ARRAYAGG(name)
JSON_OBJECTAGG()	Aggregates key-value pairs into a JSON object	JSON_OBJECTAGG(id, name)

6.1 Window (Analytic) Functions - Bonus

Function	Purpose
ROW_NUMBER()	Assigns a unique sequential number to rows within a partition
RANK() / DENSE_RANK()	Assigns a rank, with/without gaps for ties
NTILE(n)	Divides rows into n roughly equal buckets
LAG() / LEAD()	Accesses a prior/next row's value without a self-join
FIRST_VALUE() / LAST_VALUE()	Returns the first/last value in an ordered window
SUM() / AVG() / COUNT() OVER (...)	Aggregate functions used as window functions with OVER()

SECTION 7

Quick Tips & Common Patterns

Standard SELECT order

```
SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT ...
```

Insert-or-update

```
INSERT INTO t (id, name) VALUES (1, 'A') ON DUPLICATE KEY UPDATE name = VALUES(name);
```

Common Table Expression

```
WITH recent AS (SELECT * FROM orders WHERE order_date > '2026-01-01') SELECT * FROM recent;
```

Safe division

```
SELECT total / NULLIF(count, 0) FROM stats; -- avoids divide-by-zero errors
```

Date filtering

```
WHERE order_date BETWEEN '2026-01-01' AND '2026-12-31'
```

Pattern search

```
WHERE name LIKE 'A%' -- starts with A
```

Ranking within groups

```
RANK() OVER (PARTITION BY dept ORDER BY salary DESC)
```

End of Reference — Keep this handy while practicing SQL queries and building your MySQL roadmap.